

CHROME EXTENSION · MANIFEST V3

Find in Images

Making scanned PDFs searchable with Chrome's own Ctrl+F, through on-device OCR and an invisible text layer written into a copy of the PDF.



Technical Documentation & Design Rationale

JavaScript ES6+

PDF.js

Tesseract.js

pdf-lib

Manifest V3

Version 2.0 · Architecture, implementation & phase-by-phase build narrative

Contents

1 Introduction

2 Problem Statement

3 Goals, Scope & Constraints

3.1 Goals

3.2 Constraints

3.3 Non-goals

4 Functionalities & User Stories

5 System Architecture

5.1 Pipeline overview

5.2 Component map

5.3 Request sequence

6 Technologies & Integration

6.1 How the pieces talk

6.2 Vendored libraries

7 Implementation Walkthrough

7.1 Activation, badge, and restore

7.2 Download, validation, identity

7.3 Per-page processing

7.4 OCR rendering and preprocessing

7.5 Mapping words into the PDF

7.6 Post-processing for findability

7.7 Caching

7.8 Serving the copy

7.9 Resource lifecycle

8 How It Was Built: Phase by Phase

8.1 The two reversals

8.2 Release hardening (Phase 16)

9 Testing & Release

9.1 End-to-end test in real Chrome

9.2 Packaging and publishing

10 Limitations & Future Work

10.1 Limitations

10.2 Future work

1 Introduction

Find in Images is a Chrome (Manifest V3) extension that makes scanned PDFs searchable with **Chrome's own Ctrl+F**. Open a scanned PDF, press `Ctrl+Shift+F`, and a moment later the browser's normal find bar locates words that exist only as pixels inside the page images.

The extension adds **no interface of its own**. It doesn't replace Chrome's PDF viewer, draw overlays, or bring its own find bar. Instead it uses the "searchable scan" technique that document scanners use. It reads the text in the page images with on-device OCR, writes that text into a **copy** of the PDF as an invisible layer, and shows the copy in Chrome's built-in viewer. Because the text is genuinely in the file, Chrome's find, highlighting, selection and copy all work on it unchanged.

Design philosophy. Lean on native browser capabilities instead of reinventing them. We never implement search, highlighting, or a viewer — we make the document searchable and let Chrome do the rest. Everything runs locally; no document ever leaves the machine.

This document explains the problem, the architecture, every technology used and how it is integrated, a walkthrough of the implementation, and the **phase-by-phase reasoning** behind how the system was built. That history includes two reversals: from a custom viewer to an on-demand one, and from any custom viewer to none. It ends with how the extension is tested and released.

2 Problem Statement

PDFs and document images fall into two broad categories:

- **Text-based** — the characters are encoded as text. The browser can already select and search them.
- **Image-based** — scanned pages, photographed documents, screenshots, or "text" that has been flattened into a picture. To a computer these are just coloured pixels: **not selectable, not searchable**.

Chrome's built-in PDF viewer renders image-based PDFs perfectly but cannot find a single word inside them, because there is no text to find. For anyone working with scanned reports, lecture notes, invoices, or archival material, this is a daily friction point: the information is visible but not reachable.

A further subtlety: many real PDFs are **mixed**. A single page may contain native text *and* images that themselves contain text (a scanned figure with a caption, a table rendered as a picture, a logo with a tagline). A useful solution must handle native text, image text, and the two together on the same page — without duplicating words or misaligning them.

And a constraint that shaped the final design: Chrome's PDF viewer is a sealed component. Extensions cannot script it, inject into it, or add a text layer to it. Anything that keeps the native viewer must therefore change the *file* the viewer shows, not the viewer.

The problem in one line. Make the text that lives *inside images* findable through Chrome's native `Ctrl+F`, in Chrome's native viewer, without altering the original file or how it looks.

3 Goals, Scope & Constraints

3.1 Goals

- **Accurate, findable image text.** The headline feature: text inside images must be reliably found by `Ctrl+F`. This is the project's top priority.
- **Native experience, unchanged.** Chrome's own viewer, toolbar, find bar, highlights and right-click menu — nothing to learn, nothing that looks different.
- **One reversible gesture.** `Ctrl+Shift+F` (or the toolbar icon) makes the open PDF searchable; the same gesture brings the original back.
- **Works where PDFs are.** Links without `.pdf` in the address, sites the user is signed in to, and local files.
- **Responsive.** The tab stays usable while OCR runs in the background; progress is visible on the toolbar icon.
- **Offline and private.** No network calls beyond re-downloading the PDF itself; OCR and language models run entirely on-device.
- **Selectable and copyable text.** Recovered text can be selected and copied like any other PDF text.

3.2 Constraints

Hard constraint 1 — the original PDF is never modified. The source file is downloaded, read, and left untouched. Only a *copy* receives the invisible text layer, and invisible text paints nothing, so the copy looks identical.

Hard constraint 2 — Chrome's own viewer is the only UI. No custom viewer, overlay, or find bar. A Chrome look-alike viewer was built and rejected: "same behaviour ... same design and view" means the real thing, not an imitation.

Together these rule out every overlay-based design and lead directly to the searchable-copy architecture in Section 5.

3.3 Non-goals

- Editing, annotating, or re-saving PDFs for the user.
- Server-side or cloud OCR (everything is on-device by design).
- Perfect OCR. OCR is inherently imperfect; the system is designed to *maximise findability despite* recognition errors (Section 7.6) rather than to guarantee character-perfect transcription.
- Searching images on ordinary web pages. The extension has no content scripts and never touches web pages.

4 Functionalities & User Stories

The behaviour is best understood as a set of user stories, each mapped to the capability that satisfies it.

User story	How the system satisfies it
As a user , I want <code>Ctrl+F</code> to find words that are part of an image.	Image pages are OCR'd and the words are written into a copy of the PDF as invisible text that Chrome's own find matches and highlights (7.3–7.5).
As a user , I want everything to look and work exactly as before.	The copy is shown in Chrome's native viewer; invisible text paints nothing; the tab title stays the same (7.8).
As a user , I want one gesture to turn it on and off.	<code>Ctrl+Shift+F</code> or the toolbar icon starts processing; on a copy, the same gesture restores the original URL (7.1).
As a user , I want to know what is happening.	The toolbar badge shows <code>...</code> , then a percentage, then <code>✓</code> ; failures show a red <code>!</code> whose tooltip explains why (7.1).
As a user , I want it to work on any PDF I open — even from links without ".pdf", sites I'm signed in to, or my own files.	Any web or file URL is accepted; the file is re-downloaded with the user's cookies and checked for the <code>%PDF-</code> signature (7.2).
As a user of mixed PDFs, I want image text added without duplicating existing text.	OCR words that the page already contains at the same spot are dropped (7.5).
As a user , I don't want to re-pay the OCR cost every time.	Recognised words are cached in IndexedDB by a SHA-256 of the file, per OCR language (7.7).
As a non-English user , I want OCR in my language.	English, Danish, or both, chosen from the toolbar icon's right-click menu; bundled, offline language data (7.4).
As a user with faint or blurry scans, I still want text found.	Pages are rendered at ~3000 px and passed through a gain-capped contrast stretch and an unsharp mask before OCR (7.4).
As a user , I don't want an extension that slows my browser down.	OCR runs in a hidden document on a worker pool; workers, copies, and cache entries are released when no longer needed (7.9).

5 System Architecture

The extension is a **pipeline** that turns a scanned PDF into a searchable copy of itself and lets Chrome show that copy. The tab keeps showing the original while a hidden offscreen document does the work. When it finishes, the tab is navigated to the copy, which the extension's service worker serves from Cache Storage, so Chrome opens it in its own viewer.

5.1 Pipeline overview

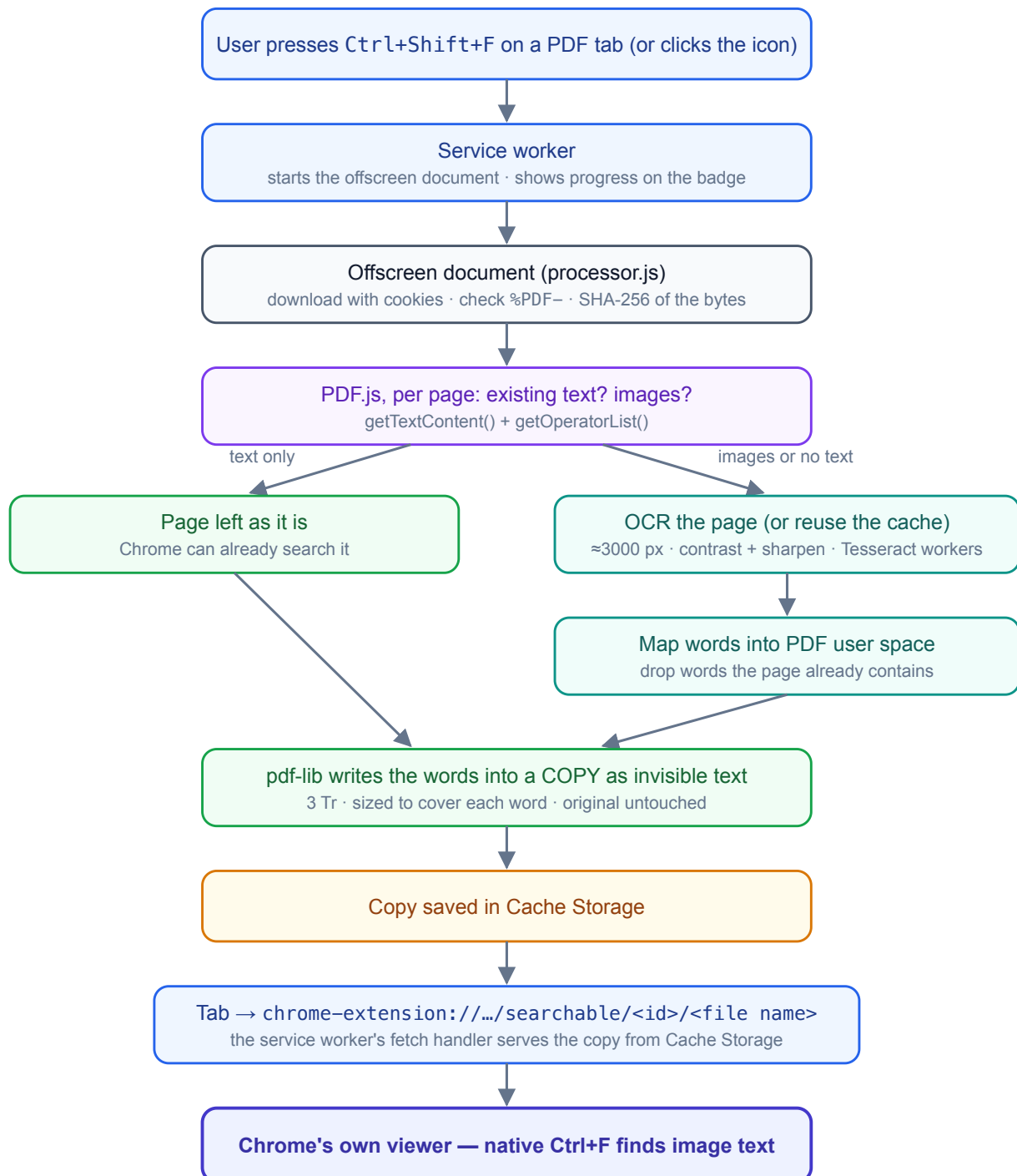


Figure 1 — End-to-end pipeline. Pages that already have text are left alone; pages with images are OCR'd, and the recognised words are written into a copy as invisible text. The copy reaches Chrome's native viewer through an extension URL.

5.2 Component map

The codebase is organised in layers: Chrome itself (unchanged), the extension runtime, the hidden offscreen document that holds the pipeline, the vendored libraries, and the browser APIs they build on.

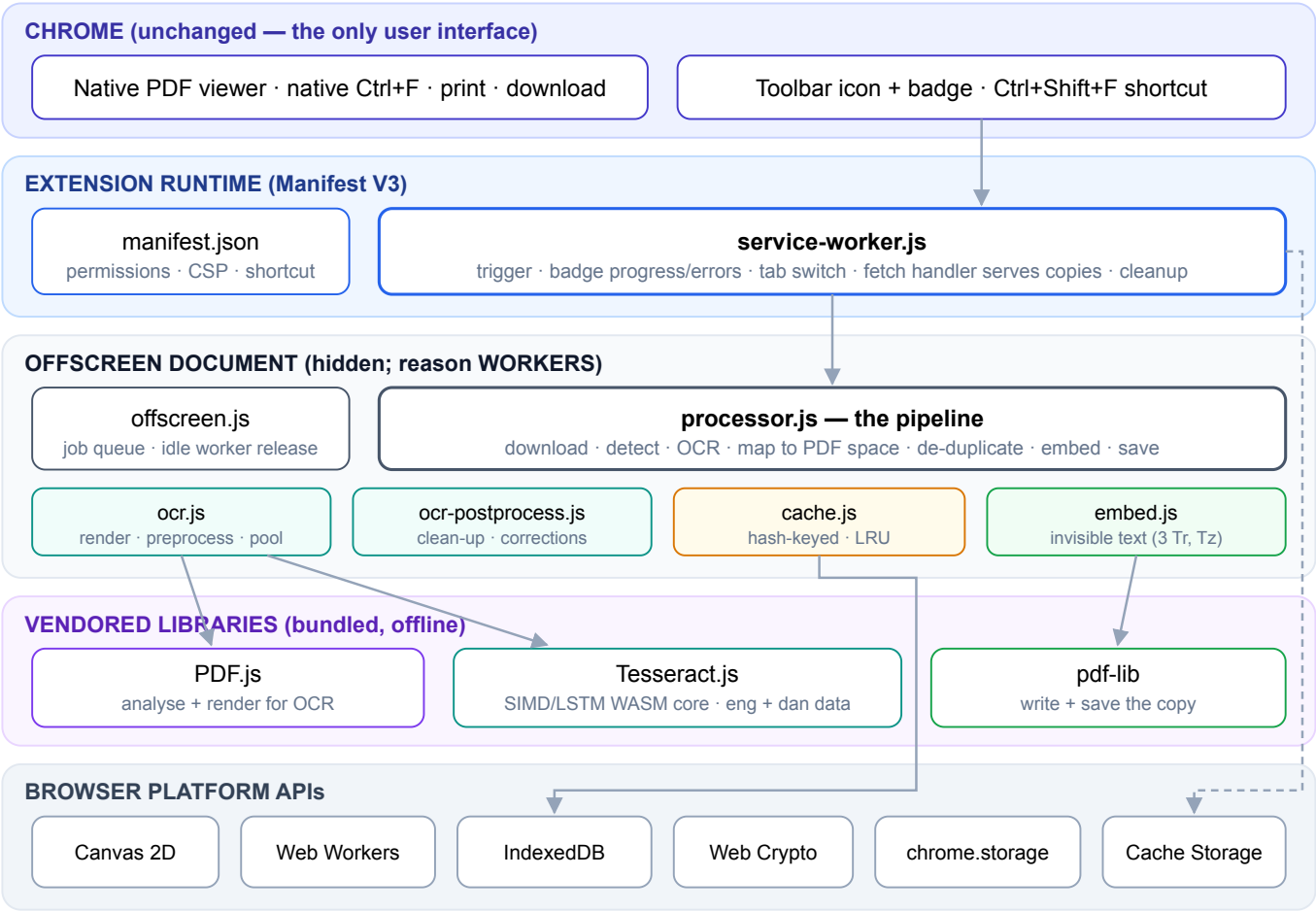
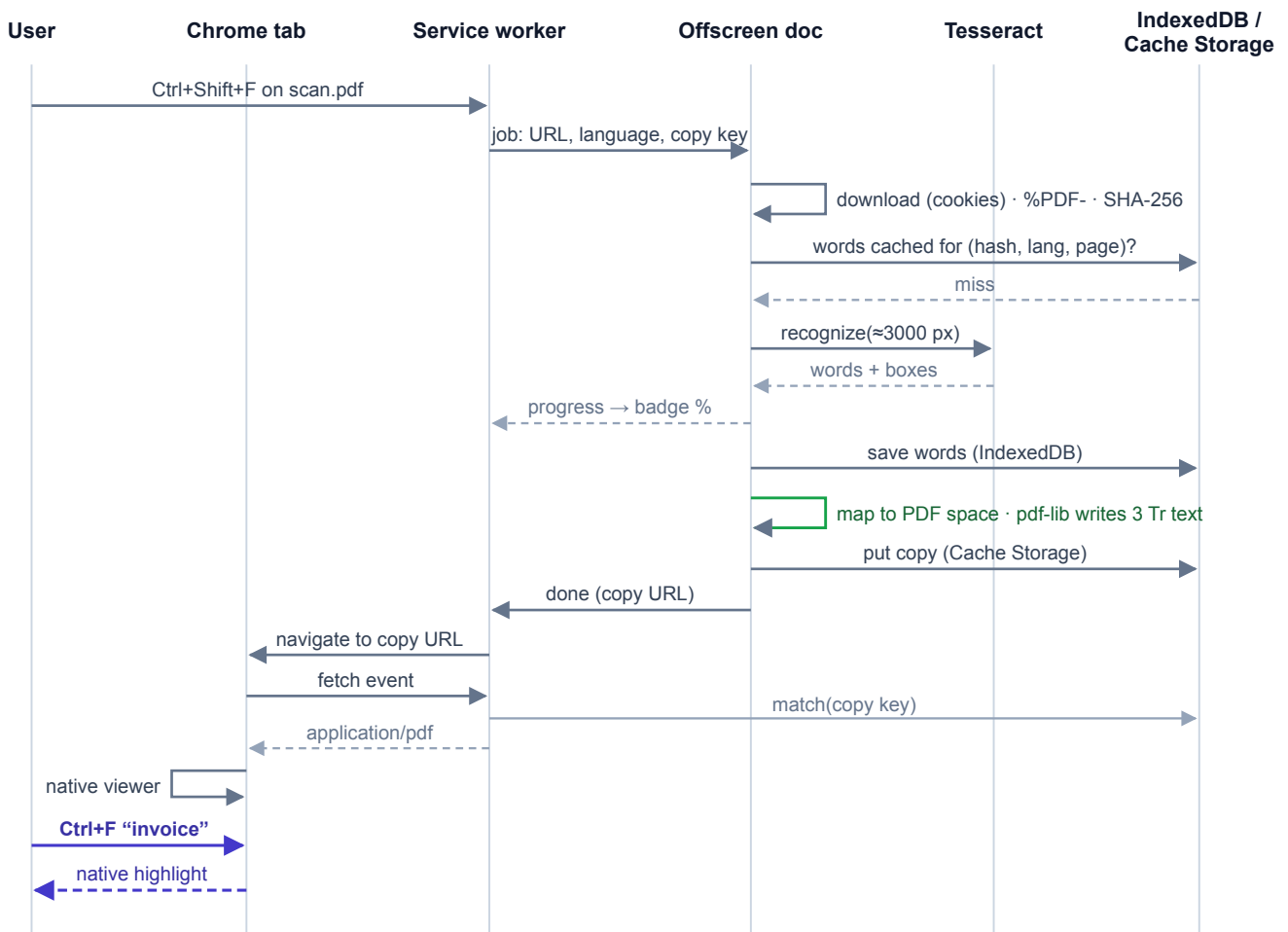


Figure 2 — Component map. The service worker talks to Chrome; processor.js in the offscreen document drives the libraries; the copy is served straight from Cache Storage.

5.3 Request sequence



On a second visit the cache hits at the lookup step and the OCR round-trip is skipped.

Figure 3 — What happens between the shortcut and the first native search, including the cache lookup that makes a second visit fast.

Three design points explain the shape of the system:

- **Change the file, not the viewer.** Chrome's viewer cannot be extended, so the only way to keep it is to give it a file that already contains the text.
- **Serve the copy from the extension's own origin.** The first version opened the copy as a `blob: URL`. In real Chrome that navigation silently never completes, because an extension's blob URL can't be opened from an ordinary website tab. The copy URL `chrome-extension://<id>/searchable/<token>/<file name>?src=<original>` works, keeps the tab title (it ends in the original file name), makes restoring trivial, and lets an expired copy fall back to the original.
- **Do the heavy work out of sight.** Rendering and OCR need a DOM and Web Workers, which a service worker lacks, and must not disturb the tab. An offscreen document provides both.

6 Technologies & Integration

Every part of the extension runs on the client. Each technology has one job; the value is in how they are coordinated.

Technology	Role	Where
Chrome Extension (Manifest V3)	Permissions (<code>offline</code> , <code>storage</code> , <code>contextMenus</code> , host access), CSP with <code>wasm-unsafe-eval</code> , shortcut, Chrome ≥ 116	<code>manifest.json</code>
Service worker + <code>chrome.action</code> / <code>chrome.commands</code>	Trigger, per-tab badge progress and errors, tab switch, <code>fetch</code> handler that serves copies, cleanup	<code>service-worker.js</code>
Offscreen document (<code>chrome.offscreen</code>)	Hidden page with DOM, canvas and Web Workers that hosts the pipeline	<code>offscreen/offscreen.js</code>
PDF.js	Per-page text and image detection; rendering pages for OCR; mapping pixels back to PDF space	<code>processor.js</code> , <code>ocr.js</code>
Canvas 2D	OCR raster target and pixel-level preprocessing	<code>ocr.js</code>
Tesseract.js (WASM) + Web Workers	OCR with word boxes on a worker pool; English and Danish data bundled	<code>ocr.js</code>
pdf-lib	Writes the invisible text into the copy and saves it	<code>embed.js</code> , <code>processor.js</code>
Web Crypto	SHA-256 of the file as its identity	<code>cache.js</code>
IndexedDB	OCR results per page, least-recently-used pruning	<code>cache.js</code>
Cache Storage	Searchable copies, served at extension URLs	<code>offscreen.js</code> , <code>service-worker.js</code>
<code>chrome.storage</code> , <code>chrome.contextMenus</code>	OCR language; which tab shows which copy; the language menu	<code>service-worker.js</code>
Chrome's native PDF viewer	All display, search, highlighting, selection, printing	Chrome

6.1 How the pieces talk

- **Service worker ↔ offscreen document — small messages only.** In: `fii-process` with the PDF URL, language, and the copy URL and cache key the service worker chose. Out: `fii-progress` (page counts, shown as a badge percentage), then `fii-done` with the copy URL or `fii-error` with a readable message.

- **Bytes travel through storage, not messages.** The offscreen document puts the finished PDF into Cache Storage; the service worker's `fetch` handler reads it from there when Chrome requests the copy URL. Both can reach it because they share the extension's origin.
- **Cache Storage keys.** Cache Storage only accepts http(s) request keys, so a copy at `chrome-extension://<id>/searchable/...` is stored under `https://find-in-images.invalid/searchable/...` (same path, placeholder origin).
- **No content scripts, no web-accessible resources.** The extension never runs code in web pages; navigation to the copy is done by the extension itself (`chrome.tabs.update`).

6.2 Vendored libraries

All libraries are bundled under `lib/` so nothing is fetched at runtime. Tesseract.js normally picks one of six WebAssembly builds at runtime. Its relaxed-SIMD build crashes on some Chrome and Apple Silicon combinations, so `lib/tesseract/worker.min.js` is patched to never choose it. Only the two builds that can actually load (SIMD + LSTM, and plain LSTM as a fallback) are shipped, which keeps the package at about 8.5 MB.

7 Implementation Walkthrough

7.1 Activation, badge, and restore

The toolbar icon and the shortcut (`Ctrl+Shift+F` ; on Mac `Control+Shift+F`, `⌘⇧F`) run one handler, `toggle(tab)` :

- **On a copy** (`chrome-extension://.../searchable/...`): read the original URL from the `src` parameter, delete the copy, navigate back.
- **On anything that isn't a web or file URL**: red badge, "open a PDF in this tab first".
- **On a `file://` URL without file access**: red badge plus the extension's settings page, where "Allow access to file URLs" lives.
- **Otherwise**: start the offscreen document if needed, choose the copy URL and cache key, and send the job.

The badge is per tab: `...` while starting, a percentage while pages are processed, `✓` when done. Errors use a red `!` for 15 seconds, and the tooltip carries the explanation. Chrome clears a tab's badge whenever the tab navigates, so the success badge is set only after the copy has finished loading. A generation counter keeps a delayed reset from wiping a newer badge.

Why `Control+Shift+F` on Mac? Chrome leaves a suggested `Cmd+Shift+F` unassigned (`chrome.commands.getAll()` reports an empty shortcut, and `chrome://extensions/shortcuts` shows "Not set"). `MacCtrl+Shift+F` is assigned, and gives the same `Ctrl+Shift+F` on every platform.

7.2 Download, validation, identity

The offscreen document downloads the tab's URL again with `credentials: "include"` , so signed-in sites return the same file the tab shows. Because many PDFs live at addresses without ".pdf", validation happens on the bytes: the `%PDF-` signature must appear in the first kilobyte. HTML responses are reported as "not a PDF (or a sign-in page)".

The bytes are loaded by PDF.js (analysis and rendering) and pdf-lib (writing), and hashed with SHA-256, all in parallel. Encrypted files are reported clearly, because pdf-lib cannot write into them. pdf-lib's error class is transpiled ES5, so `instanceof` fails on it and the check matches the message instead. The PDF.js loading task is destroyed in a `finally` block, so a long-lived offscreen document never accumulates documents.

7.3 Per-page processing

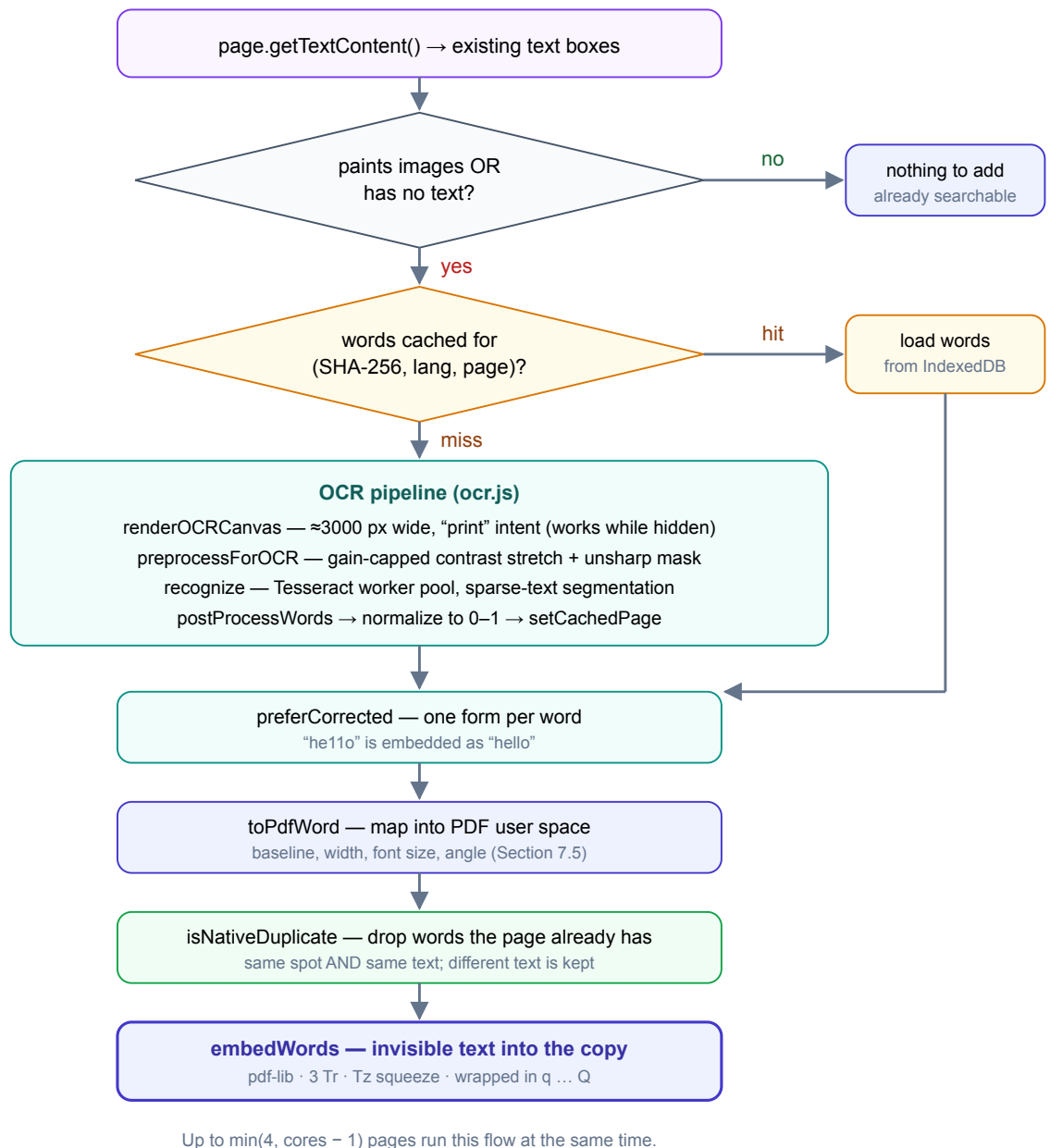


Figure 4 — What happens to each page. Up to $\min(4, \text{cores} - 1)$ pages run this flow concurrently.

A page is OCR'd when it paints raster images (found in its operator list) **or** has no extractable text at all. Image detection fails open: if the operator list can't be read, the page is OCR'd anyway.

7.4 OCR rendering and preprocessing

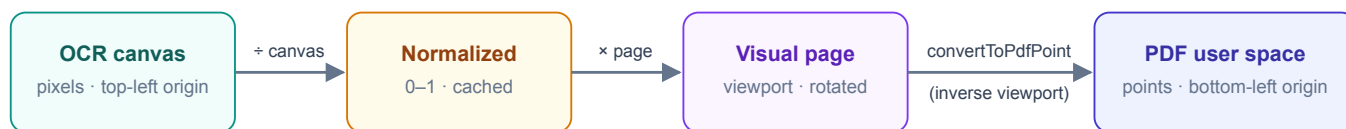
The page is rendered to an offscreen canvas about 3000 px wide with PDF.js's "print" intent. The default intent is paced by `requestAnimationFrame`, which never fires in a hidden document. The image is then preprocessed:

```
gray = 0.299 R + 0.587 G + 0.114 B
lo, hi = 2nd and 98th percentile of the histogram
R = max(96, hi - lo) # gain floor: at most ~2.7x
v' = 255 - 255 · (hi - v) / R # anchored at the white point
sharp = v' + (v' - boxBlur(v', r=2)) # unsharp mask, 0(n) sliding window
```

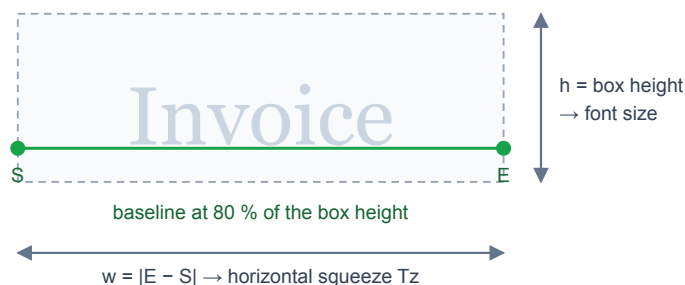
The floor on `R` fixed a real failure. On a mostly blank page, text is under 2% of the pixels, so the "2nd percentile" is background too. The uncapped stretch multiplied contrast by up to 127x, turned JPEG noise into black speckle, and made one test page unreadable.

Recognition uses a Tesseract.js scheduler over `min(4, cores - 1)` LSTM workers per language, with sparse-text segmentation (better for tables and scattered text). Workers are terminated 60 s after the job queue empties and recreated on demand.

7.5 Mapping words into the PDF



One recognized word → one invisible text object



```

q BT
3 Tr           % invisible
/F h Tf       % size = h
s Tz          % s = 100·w/w_nat
cos sin -sin cos Sx Sy Tm
(Invoice) Tj
ET Q
  
```

Chrome draws its find highlight from this geometry, so it lands exactly on the visible word — at any zoom and on rotated pages.

Figure 5 — From an OCR pixel box to one invisible text object in PDF user space.

Word boxes are normalized to 0–1 (so cached results don't depend on render size), scaled to the page viewport, and mapped back into PDF user space with PDF.js's inverse viewport transform (`convertToPdfPoint`). That transform already includes page rotation. Four mapped points give everything needed:

- baseline start **S** and end **E** at 80% of the box height → width $w = |E - S|$ and angle $\theta = \text{atan2}(E - S)$;
- box top and bottom → height h , used as the font size.

Each word becomes `BT 3 Tr /F h Tf s Tz [cosθ sinθ -sinθ cosθ Sx Sy] Tm (word) Tj ET` , with $s = 100 \cdot w / w_{\text{natural}}$ clamped to 1–1000. Chrome's find highlight is drawn from exactly this geometry, so it covers the visible word at any zoom. The block is wrapped in `q ... Q` so its text state can't leak into the page.

De-duplication. An OCR word is dropped when the midpoint of its baseline lies inside an existing text item's box *and* that item's text contains the word. Different text at the same spot (a caption baked into a figure next to real text) is kept.

7.6 Post-processing for findability

Native find is exact-match, so the stored text has to be right. Post-processing drops near-zero-confidence noise and trims non-letter junk from word edges. The trimming uses Unicode letter classes, so Danish "på" stays "på". It also corrects likely digit/letter confusions in mostly-letter words (`he11o` → `hello` , `ca5h` → `cash`). The old DOM viewer injected both forms; inside a real PDF two words at one spot would double Chrome's match count and duplicate copied text, so the copy contains **one** form, the corrected one.

7.7 Caching

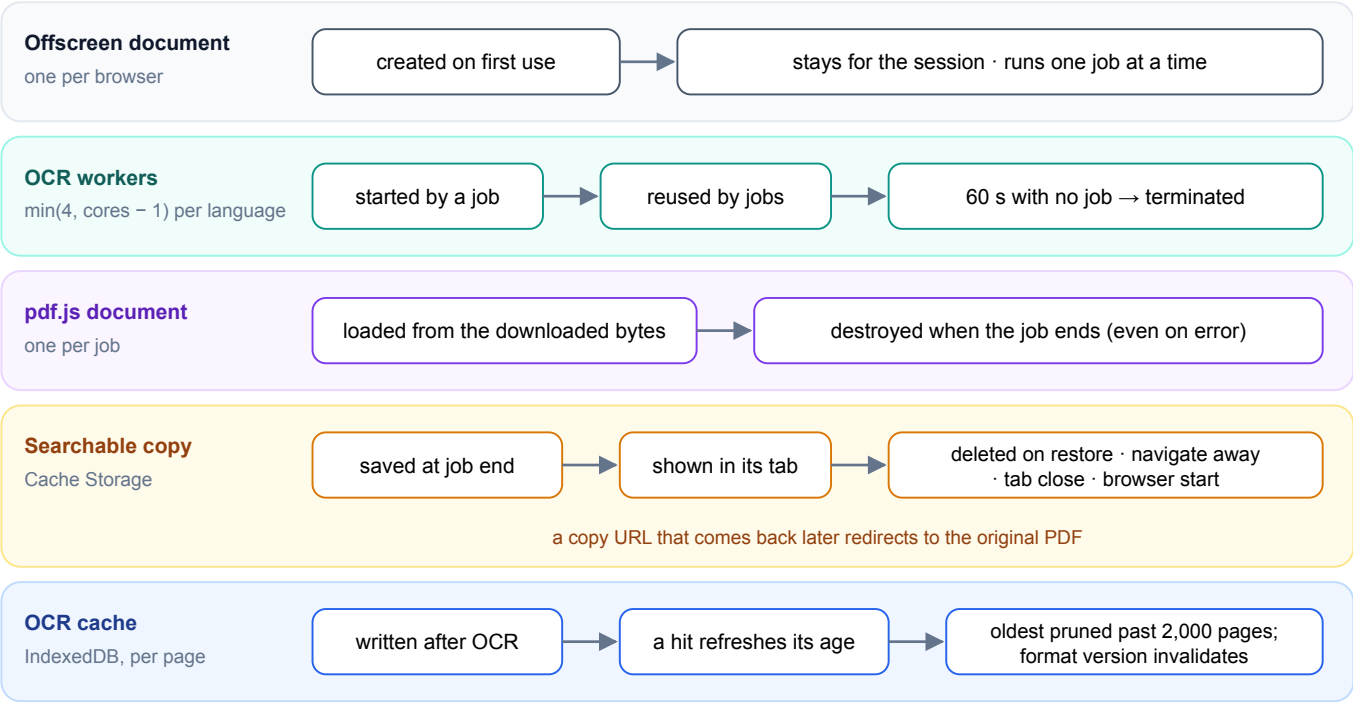
Words are stored per page in IndexedDB under `<sha256>::<lang>::<page>` , with a format version (bumped whenever the pipeline changes) and a last-used timestamp. Keying by content means the same file reached through two URLs is recognised once, and a changed file at a familiar URL never gets stale words. Empty results are cached too. After every job, the oldest records are deleted until at most 2,000 pages remain.

7.8 Serving the copy

```
self.addEventListener("fetch", (event) => {
  const url = new URL(event.request.url);
  if (url.origin !== self.location.origin || !url.pathname.startsWith("/searchable/")) return;
  event.respondWith(serveCopy(url));
});
async function serveCopy(url) {
  const hit = await (await caches.open(COPY_CACHE)).match(COPY_KEY_ORIGIN + url.pathname);
  if (hit) return hit;
  const src = url.searchParams.get("src");          // copy expired → back to the original
  if (src && isFetchableUrl(src)) return Response.redirect(src, 302);
  return new Response("This searchable copy is no longer available.", { status: 404 });
}
```

Before switching, the service worker checks that the tab still shows the same PDF (the user may have moved on). Chrome receives an `application/pdf` response and opens its own viewer. The URL's last segment is the original file name, so the tab title is unchanged.

7.9 Resource lifecycle



Every resource has an owner and an end — nothing grows for the lifetime of the browser.

Figure 6 — Every resource the extension creates, and what ends it.

8 How It Was Built: Phase by Phase

The extension was built one phase at a time, each shipped and tested before the next. Each phase answered the question the previous one raised.

#	Phase	What it set out to solve	What it revealed
1	Extension skeleton & PDF interception	Chrome's viewer can't be scripted, so redirect <code>.pdf</code> navigations to our own viewer	A viewer that shows nothing yet
2	PDF.js rendering	Render pages to canvas inside the extension (bundled for MV3's CSP)	Pages display but nothing is searchable
3	Detection, OCR, invisible overlay	Native text + Tesseract OCR, injected as invisible DOM spans; switched from <i>either/or</i> to <i>both</i> with de-duplication	OCR re-runs on every load
4	IndexedDB cache	OCR once; normalized coordinates survive any zoom	Language data still came from a CDN
5	Fully offline	Bundle the language models	—
6	Correction variants	Exact-match find vs. OCR misreads (<code>he11o</code>)	Rotated pages misaligned
7	Rotated pages	Angle and size from the transform matrix	Non-English documents
8	Multi-language OCR	English / Danish, cached per language	Large PDFs were slow
9	Performance	Lazy rendering, idle-time indexing, parallel OCR workers	Selection and native find looked wrong
10	Readability & search UX	Measure-and-scale spans; custom find bar	Tables, small and faint text still missed
11	OCR preprocessing	3000 px OCR canvas, sparse segmentation, contrast + sharpening	—
12	Activation on demand	Stop hijacking every PDF; <code>Ctrl+Shift+F</code> opens the OCR viewer only when wanted	Still a different viewer
13	Imitating the native viewer	Chrome-style toolbar, find bar and highlights; phrase search across words; hidden-tab rendering fix	The user's verdict: an imitation is not the original
14	Native viewer only	Delete the viewer; write OCR text <i>into a copy</i> (sandwich PDF) and show it in Chrome's viewer	The tab switch was never tested in real Chrome
15	First use in the user's browser	Relaxed-SIMD Tesseract core crashed → patched out; Mac shortcut showed "Not set"	—
16	Release hardening in real Chrome	End-to-end tests of the packaged extension; fixes below	Ready to publish

8.1 The two reversals

From automatic to on demand (Phase 12). Intercepting every PDF replaced Chrome's viewer even for text PDFs that were already searchable. That threw away the native toolbar, thumbnails, printing, Google Lens and right-click menu while adding nothing. The extension's value is real only for scanned PDFs, so it became opt-in.

From a custom viewer to none (Phases 13–14). The on-demand viewer was then made to look like Chrome's, down to the find bar and highlight colours. The user's requirement was sharper: "same behaviour as that of original ... and same design and view also", with no different view and no change to the PDF's format. No imitation can meet that. Giving Chrome a *file* that contains the text can, so about 1,300 lines of viewer code were deleted and replaced by the searchable-copy pipeline. The original file is still never modified, and the copy's visible content was verified identical by pixel comparison (0 of 2,298,825 pixels differed).

8.2 Release hardening (Phase 16)

The first automated test of the *packaged* extension in real Chrome showed that the core feature had never worked. OCR completed and the copy existed, but Chrome silently refused to navigate a website tab to the extension's `blob:` URL. That led to the extension-origin delivery of Section 7.8. The same review and test suite also found:

- a release zip missing the `offscreen/` folder (the packaging script still listed the deleted viewer);
- the contrast-stretch failure on sparse pages (Section 7.4);
- the Mac shortcut never being assigned (Section 7.1);
- PDFs without ".pdf" in the URL being ignored;
- a URL-keyed cache that could apply old words to a changed file;
- Danish letters being stripped from word edges ("på" stored as "p");
- silent failures for encrypted PDFs, non-PDF pages and disabled file access, and a success badge Chrome erased on navigation;
- unbounded resources: pdf.js documents and OCR workers never released;
- store requirements: an over-long manifest description, a privacy policy, and listing material.

Lesson. Every component had been tested, yet the product didn't work. The hardest late problems weren't algorithms but platform behaviour — what Chrome lets an extension do with blob URLs, shortcuts and badges — and they only appeared with the real browser in the loop.

9 Testing & Release

9.1 End-to-end test in real Chrome

```
npm run test:e2e
```

The test builds the release zip, unpacks it, and launches Google Chrome with a throwaway profile over the DevTools protocol (`--remote-debugging-pipe`). It loads the extension with `Extensions.loadUnpacked` , which branded Chrome still allows for automation. It then attaches to the extension's service worker and calls the same `toggle()` the toolbar icon calls. A local HTTP server plays the web, and each check observes what Chrome actually did (tab URL and title, toolbar badge). The copy is read back out of Cache Storage and its text extracted with an independent `PDF.js`.

Scenario	Verified
Shortcut	Chrome actually assigns <code>Ctrl+Shift+F</code> (^⇧F on Mac)
Scanned PDF	Tab switches to the copy; copy's text has the page's words; same page count and tab title; success badge
Reload / restore	Reload keeps the copy; the gesture restores the original and deletes the copy
URL without ".pdf"	Processed; tab title kept; OCR cache reused across URLs
Expired copy	Reloading after the copy is gone lands on the original
Signed-in PDF	Cookie-protected PDF processed; closing the tab deletes its copy
Changed file, same URL	New file's words only
Encrypted PDFs	Open-password and permissions-only PDFs report a clear error
Non-PDF page	"Not a PDF" error
Local file	Processed and restored (with file access on)
Idle worker release	OCR still correct after the workers were released

Fixtures (`test/e2e/fixtures/`) are US Letter pages rendered at 300 DPI and saved as image-only PDFs, like scanner output. On them, OCR found all 74 words of the dense page (one character misread) and read the other two word for word, tables included. A one-page scan takes about 2–2.5 s from shortcut to searchable copy on the development Mac, about 0.5 s when cached.

`CHROME=/path/to/chrome` overrides the browser location; `HEADFUL=1` shows the window.

9.2 Packaging and publishing

```
npm run package    # → dist/find-in-images-<version>.zip
```

The zip contains `manifest.json` , `service-worker.js` , `icons/` , `lib/` and `offscreen/` (about 8.5 MB). Publishing on the Chrome Web Store needs a developer account, the privacy policy (published with this documentation at `/privacy.html`), screenshots, and a justification for each permission. `store/listing.md` contains all of it, ready to paste.

10 Limitations & Future Work

10.1 Limitations

- **OCR isn't perfect.** Native find is exact, so a misread character can hide a word; correction variants only cover common digit/letter confusions.
- **Whole document first.** The copy appears when every page is done; large scans take a while (the badge shows progress).
- **Encrypted PDFs** can't be written to, so they are reported and left alone.
- **Latin characters only.** The invisible text uses Helvetica's WinAnsi encoding: fine for English and Danish, not for Greek, Cyrillic or CJK.
- **The PDF must be downloadable again.** PDFs from form submissions or one-time links can't be fetched a second time; PDFs inside another site's viewer aren't PDF tabs.
- **The address bar changes** to `chrome-extension://.../searchable/...` while the copy is shown — the one visible sign of the switch.
- **Broad host access.** Downloading a PDF from wherever it lives needs access to all sites; it is used only when the user invokes the extension.
- **Large type at the OCR resolution.** The 3000 px render suits normal text sizes; on pages with unusually large type, Tesseract can skip bordered table cells.
- **Small evaluation.** A few clean, synthetic scans on one machine and one Chrome version.

10.2 Future work

- Process pages in reading order and offer a partial copy early for long documents.
- Embed a subsetted Unicode font for non-Latin scripts; more languages; automatic script detection.
- Dictionary-based correction, skew, and OCR at more than one scale.
- Ask for access to a site only when the user invokes the extension there, once verified that this covers downloading the PDF.
- A benchmark on real scans (character and word error rates, per-page latency).